Notifications — Plaintext Password Leaks Across Email & SMS

InstantMudra Enterprises Admin Portal

⚠ CRITICAL — new user's plaintext password persisted to DB and written to the log file

OnboardingServiceImpl — welcome-email flow, on every new user created

```
vars.put("password", randomPassword);
String emailBody = emailService.buildEmailFromDB(EmailType.ONBOARD_USER, vars); // real password substituted in
emailService.sendSimpleEmail(credentials, fromEmail, fromPassword, email, subject, emailBody);
```

→ EmailServiceImpl.sendSimpleEmail() logs request = "To: ...\nSubject: ...\nBody: " + emailBody
→ emailLogRepository.save(emailLog) — the plaintext password is now a permanent row in the EmailLog table

```
log.info("SMS (mock) sent to mobile: " + mobileNo + " with password: " + randomPassword); // ALSO in the app log
```

Developer's own comment on the line returning this password: "For testing: return plain password (REMOVE THIS IN PRODUCTION!)"

⚠ CRITICAL — SMSGatewayAPI (Gupshup fallback): hardcoded credential + logged OTP

```
String fullUrl = "https://enterprise.smsgupshup.com/GatewayAPI/rest?userid=2000197160&password=IMudra@2022&" + data;
```

— the Gupshup account password, hardcoded directly in Java source, not application.properties

```
String requestLog = fullUrl; // includes msg=<urlencoded OTP text> AND the password above
commonFunctions.saveLoginLogs(requestLog, requestDateTime, response, responseDateTime, null);
```

→ every OTP sent through this fallback, plus the gateway password, is persisted to the database in plaintext

Contrast — ShivSmsService (the primary provider) does neither

Its gateway password comes from @Value("${sms.shiv.password}") — application.properties, not hardcoded in source (still plaintext config, per Module 1, but not committed inline in a .java file).

No saveLoginLogs() / request-URL logging anywhere in this class — OTP content and the gateway password are never persisted. A rare case in this codebase where the fallback path is the worse-built one, not the primary.

Minor findings

WebSocketService — old version left as a comment

An earlier, superseded version of sendToAll/sendToRole/sendToUser (single-channel, no "channel" parameter) sits commented out directly above the live multi-channel versions — the same pattern seen in Modules 2 and 9.

Exception handling & mail sender setup

SMSGatewayAPI and EmailServiceImpl both catch Exception and call e.printStackTrace() rather than the SLF4J logger used elsewhere. EmailServiceImpl builds a new JavaMailSenderImpl per call with a hardcoded SMTP host.

Correctly implemented, by contrast Minor / dead code Critical — confirmed plaintext-credential leak